

A Practical Spec-Driven Development Framework

A step-by-step, lightweight workflow for building a multi-tenant dental clinic SaaS with AI coding agents

ONE-TIME FOUNDATION	REPEAT PER FEATURE
Product brief System rules Architecture Roadmap	Spec Tasks Implementation Verification

Core rule: define only what prevents ambiguity, security mistakes, or expensive rework. Detail the current feature, not the entire future product.

1. How to Use This Guide

Use the foundation steps once at the beginning. Then repeat the feature cycle for one vertical slice at a time. Do not fully specify future features before their turn.

Stage	Output	Done when
1. Product	docs/01-product.md	The first release, users, exclusions, and success criteria are clear.
2. System rules	docs/02-system-rules.md	Tenant, permission, offline, money, audit, and deletion invariants are agreed.
3. Architecture	docs/03-architecture.md	The main applications, boundaries, data ownership, API, and sync direction are chosen.
4. Roadmap	docs/04-roadmap.md	Features are ordered; only the next feature is moved to Spec.
5. Feature spec	features//spec.md	Scope, rules, scenarios, contracts, edge cases, and acceptance criteria are reviewable.
6. Tasks	features//tasks.md	The spec is decomposed into small, testable implementation tasks.
7. Build	Code + tests	Tasks are completed without silently changing product rules.
8. Verify	Acceptance review	The feature meets acceptance criteria, security rules, and documentation.

Recommended reading rule for an AI agent

Always read system rules and architecture. For implementation, read only the current feature's spec and tasks. Read other feature specs only when a declared dependency requires it.

2. Step 1 - Write the Product Brief

Purpose: establish what the product is and what the first release is not. Keep it to one or two pages.

File: docs/01-product.md

```
# Product

## Problem
What operational problems will the product solve?

## Users
General manager, doctor, reception, accounts owner.

## First Release
The capabilities that must be usable in the pilot.

## Out of Scope
Items intentionally delayed.

## Platforms
Web and Windows desktop now; mobile later.

## Success Criteria
Observable conditions that make the pilot useful.
```

Checklist

- The target clinic types and users are named.
- The pilot scope can realistically be delivered in the available time.
- Deferred items are explicit, so the agent does not add them accidentally.
- Success is described as user-observable outcomes, not a list of technologies.

Stop condition

Stop editing when the team can decide whether a requested feature belongs in the first release.
Technical implementation does not belong in this file.

3. Step 2 - Define System-Wide Rules

Purpose: capture invariants that every feature must obey. This is the most important shared document for a healthcare SaaS.

File: docs/02-system-rules.md

- **Tenant isolation:** every tenant-owned record is bound to a clinic; authorization is enforced on the server.
- **Roles and permissions:** define the default authority of manager, reception, doctor, and accounts owner.
- **Clinical versus financial visibility:** medical read access must not imply access to another doctor's financial data.
- **Offline and sync:** identify offline-capable operations, client identifiers, retry behavior, idempotency, and conflict policy.
- **Money:** store amounts in minor units; define currency per clinic; reverse financial records instead of silently deleting them.
- **Audit:** record actor, clinic, action, subject, timestamp, and relevant before/after information.
- **Deletion:** specify archive, soft-delete, retention, and restoration expectations for sensitive records.

Minimum invariant format

```
### Rule: Tenant Isolation
Every tenant-owned record SHALL include a clinic identifier.
The server SHALL derive or validate tenant context from the authenticated session.
A client-supplied clinic identifier SHALL NOT grant access.

Verification: automated tests attempt cross-clinic reads and writes.
```

Checklist

- Every rule is stated as a behavior that can be verified.
- Authorization is defined as a server responsibility.
- Offline retries cannot duplicate sensitive operations.
- Audit and financial correction behavior are explicit.

4. Step 3 - Record the Essential Architecture

Purpose: give the agent stable technical boundaries without designing every module in advance. Aim for two to four pages.

File: docs/03-architecture.md

- Applications: Next.js web, Node.js backend, Flutter Windows desktop, Flutter mobile later.
- Structure: modular monolith organized by business feature or bounded module.
- Data ownership: identify the server source of truth and the local offline store.
- API conventions: authentication, authorization, response shape, error shape, versioning, and idempotency.
- Synchronization: local operation queue, stable IDs, retry strategy, server acknowledgements, and conflict resolution.
- Testing boundaries: domain tests, authorization tests, API integration tests, and critical end-to-end flows.

Decision rule

Create a separate ADR only when a decision is expensive to reverse, has serious competing options, or needs a durable explanation. Do not create ADRs for routine library or naming choices.

Architecture readiness checklist

- The deployment units and responsibilities are understandable.
- Tenant isolation and authorization have an enforcement location.
- The offline source, sync direction, and conflict ownership are identified.
- The first feature can be implemented without inventing a second architecture.

5. Step 4 - Create the Feature Roadmap

Purpose: order delivery without prematurely specifying the whole system.

File: docs/04-roadmap.md

```
# Feature Roadmap

## Pilot
1. Clinic setup and authentication
2. Users, roles, and permissions
3. Patients
4. Appointments
5. Visits and treatment plans
6. Payments and debts
7. WhatsApp notifications

## Later
- Advanced inventory
- Advanced doctor accounting
- Mobile application
- Advanced analytics

## Statuses
Backlog -> Spec -> Ready -> Building -> Testing -> Done
```

Ordering rules

- Build foundational security and tenant context before business features that depend on them.
- Prefer end-to-end vertical slices over completing every backend layer first.
- Move only the next feature into detailed specification.
- Split features that cannot be reviewed or demonstrated independently within a short development cycle.

Recommended first slices

Clinic + sign-in, then users + permissions, then patient registration and listing. This sequence tests tenant isolation, role enforcement, API conventions, persistence, and basic UI early.

6. Step 5 - Specify the Current Feature

Purpose: create a small contract for the next buildable feature. Target two to five pages. Split the feature if it grows far beyond that.

File: features/spec.md

```
# Feature Name

## 1. Goal
The user value in one short paragraph.

## 2. Scope
### Included
- ...
### Not Included Now
- ...

## 3. Actors and Permissions
Who may view, create, update, approve, cancel, or archive?

## 4. Main Scenarios
Numbered user and system flows.

## 5. Business Rules
Rules the implementation must enforce.

## 6. Data and Contracts
Key fields, states, inputs, outputs, events, and errors.

## 7. Edge Cases
Offline, retry, duplicate, unauthorized, validation, and conflict cases.

## 8. Acceptance Criteria
Observable and testable completion conditions.
```

Specification quality test

- A developer can identify what is inside and outside the feature.
- Permissions are explicit for every sensitive action.
- Business rules do not depend on UI hiding.
- Acceptance criteria describe outcomes, not implementation tasks.
- Offline and cross-tenant cases are addressed when relevant.

7. Example - Patient Registration Spec

Use this level of detail as a reference, not as a universal template to copy blindly.

```
# Patient Registration

## Goal
Allow authorized clinic staff to register a patient online or offline.

## Included
Create a patient, validate required fields, warn about likely duplicates, and queue offline
creation for synchronization.

## Not Included
Full medical history, treatment planning, document upload, and patient merging.

## Permissions
The manager and reception may create patients. A doctor may create patients only if clinic policy
enables it. Every action is restricted to the active clinic.

## Rules
- Full name is required.
- Phone is optional.
- A matching phone produces a warning, not automatic rejection.
- Creation must have a stable client operation ID.
- Retrying synchronization must not create a second patient.
- Creation is written to the audit log.

## Acceptance Criteria
- Authorized staff can create a valid patient.
- Invalid required fields return field-level errors.
- Clinic A cannot read or change Clinic B's patient.
- Offline creation appears locally as pending.
- Reconnection synchronizes the patient once.
- Repeated delivery of the same operation remains idempotent.
```

Contract boundary

Add concrete request, response, error, and state definitions when the feature is ready for implementation. Avoid defining unrelated future endpoints.

8. Step 6 - Convert the Spec into Tasks

Purpose: turn decisions into small implementation units. Tasks execute the spec; they must not introduce new product behavior.

File: features//tasks.md

```
# Feature Tasks

## Domain and Data
- [ ] Add entity/state changes and migration.
- [ ] Implement business rules.
- [ ] Add unit tests for critical rules.

## Backend/API
- [ ] Implement use case.
- [ ] Enforce tenant and role authorization.
- [ ] Add endpoint and error mapping.
- [ ] Add integration tests.

## Web
- [ ] Build the user flow.
- [ ] Handle validation, loading, error, and empty states.

## Desktop/Offline
- [ ] Persist locally.
- [ ] Queue the operation and display sync status.
- [ ] Test retry and duplicate prevention.

## Verification
- [ ] Run all acceptance scenarios.
- [ ] Review tenant isolation and permissions.
- [ ] Reconcile documentation with final behavior.
```

A good task

- Produces a concrete, reviewable result.
- Is small enough to implement and test without losing context.
- Maps to a rule, scenario, contract, or acceptance criterion.
- Does not contain unresolved product decisions.

9. Steps 7-8 - Implement, Verify, and Close

Implementation loop

- Select one unchecked task.
- Read the relevant spec sections and shared system rules.
- Implement the smallest complete change.
- Run targeted tests, then the broader relevant suite.
- Mark the task complete only after verification.
- If behavior must change, update and review the spec before relying on the new behavior.

Feature completion gate

- Every acceptance criterion passes.
- Unauthorized and cross-clinic attempts are tested.
- Relevant online, offline, retry, and conflict behavior is tested.
- Financial or clinical audit requirements are satisfied.
- The implementation matches the final contracts.
- No out-of-scope behavior was added silently.
- The spec and task list reflect the delivered behavior.

Definition of Done

A feature is Done when it is demonstrable, testable, secure within its scope, and documented accurately - not merely when the interface appears to work.

10. Repository Layout and Agent Instructions

Recommended minimal layout

```
project/
|-- AGENTS.md
|-- docs/
|   |-- 01-product.md
|   |-- 02-system-rules.md
|   |-- 03-architecture.md
|   |-- 04-roadmap.md
|-- features/
|   |-- auth/
|   |   |-- spec.md
|   |   |-- tasks.md
|   |-- patients/
|   |   |-- spec.md
|   |   |-- tasks.md
|-- apps/
|-- web/
|-- server/
|-- desktop/
```

Minimal AGENTS.md

```
# Agent Instructions

1. Read docs/02-system-rules.md and docs/03-architecture.md.
2. For implementation, read only the current feature's spec.md and tasks.md unless a dependency is declared.
3. Do not expand feature scope without approval.
4. Enforce tenant isolation and permissions on the server.
5. Test acceptance criteria relevant to the task.
6. Never change a contract or business rule silently.
7. Mark tasks complete only after verification.
8. Do not read unrelated feature specifications by default.
```

Keep AGENTS.md operational. Product truth belongs in the product, system, architecture, and feature documents; do not duplicate it across instruction files.

11. The Whole Workflow on One Page

Stage	Output	Done when
Foundation	Write the four shared documents	They are short, approved, and sufficient to choose the first slice.
Select	Move one roadmap item to Spec	The item is small enough to demonstrate independently.
Specify	Write goal, scope, actors, rules, contracts, edge cases, acceptance	No blocking product decision remains.
Plan	Create tasks mapped to the spec	Tasks are ordered, testable, and bounded.
Build	Implement one task at a time	Each completed task has evidence from tests or inspection.
Verify	Run acceptance and security checks	All criteria pass; docs match code.
Close	Mark the feature Done	The feature can be demonstrated and safely used within its stated scope.
Repeat	Select the next feature	Lessons update shared rules only when truly system-wide.

Avoid these failure modes

- Writing detailed specs for the entire future product before learning from the first slice.
- Treating a long chat transcript as the source of truth.
- Allowing the coding agent to invent permissions, accounting rules, or sync behavior while coding.
- Duplicating the same rule in many documents until they disagree.
- Calling a feature Done because the happy-path UI works.

Final principle

Specify enough to remove dangerous ambiguity. Implement enough to produce a complete vertical slice. Verify against observable acceptance criteria. Then move forward.

12. Reference - Making OpenSpec Respect Project Rules

OpenSpec improves consistency, but prompt context is not an enforcement mechanism. Reliable compliance requires four layers: injected context, a pre-implementation review gate, executable tests, and CI enforcement.

Authority order

- 1. docs/02-system-rules.md
- 2. docs/03-architecture.md
- 3. openspec/specs/
- 4. Current change artifacts
- 5. tasks.md
- 6. Existing implementation

If a lower-level artifact conflicts with a higher-level source, the lower-level artifact must be corrected. The agent must never resolve product conflicts silently.

Four protection layers

Stage	Output	Done when
Context	openspec/config.yaml + AGENTS.md	Every generated artifact sees the critical invariants and mandatory references.
Review gate	READY or BLOCKED verdict	No implementation begins while rules conflict or product decisions remain unresolved.
Executable proof	Authorization, tenant, sync, and accounting tests	Violations become failing tests instead of subjective review comments.
Merge gate	CI validation	Non-compliant or untested changes cannot be merged.

Important limitation

OpenSpec project context and operation guidance influence the agent, but they are advisory. OpenSpec validation checks artifacts and structure; it does not prove that implementation behavior satisfies every business invariant.

13. Reference - Recommended OpenSpec Configuration

File: openspec/config.yaml

```
schema: spec-driven

context: |
Multi-tenant, offline-capable dental clinic SaaS.

Mandatory sources:
- docs/01-product.md
- docs/02-system-rules.md
- docs/03-architecture.md
- docs/04-roadmap.md
- AGENTS.md

Conflict precedence:
1. System rules
2. Architecture
3. Main OpenSpec specs
4. Current change artifacts
5. Existing implementation

Critical invariants:
- Every tenant-owned record belongs to a clinic.
- Tenant isolation is enforced on the server.
- Client-supplied clinicId never grants access.
- Medical access does not imply financial access.
- Financial operations are auditable.
- Offline retries are idempotent.
- Approved feature scope is not expanded.

If a change conflicts with these sources, STOP and report it.

rules:
proposal:
- State which project documents were consulted.
- List affected system-wide invariants.
- Identify scope exclusions and unresolved decisions.
specs:
- Reference applicable system rules by file and heading.
- Include authorization and tenant-isolation scenarios.
- Include offline, retry, duplicate, and conflict scenarios when relevant.
- Do not weaken an invariant silently.
design:
- Explain tenant isolation and permission enforcement.
- Identify data ownership and idempotency boundaries.
- Report architecture deviations as blockers.
tasks:
- Add tests for security-sensitive acceptance scenarios.
- Include a final specification-compliance review.

operations:
apply:
guidance:
- Read system rules, architecture, and the current change.
- Implement approved tasks only.
- Stop on unresolved product decisions.
- Run focused tests after each task.
```

14. Reference - Agent Rules and Review Gate

Minimal AGENTS.md additions

```
## Mandatory OpenSpec Workflow

Before proposing or implementing a change:
1. Read system rules, architecture, and all current change artifacts.
2. Extract the rules relevant to the change.
3. Do not edit code when behavior is ambiguous or conflicts with an invariant.
4. Never resolve tenant, permission, accounting, or sync decisions silently.
5. Report the conflicting sources, affected rule, options, and recommendation.
6. Complete work only after acceptance, authorization, tenant, and relevant offline tests pass.
```

Required review before apply

```
Review this OpenSpec change against:
- docs/02-system-rules.md
- docs/03-architecture.md
- existing openspec/specs/

Do not modify code.

Return:
1. Applicable project rules
2. Conflicts
3. Missing authorization scenarios
4. Missing tenant-isolation scenarios
5. Missing offline/idempotency scenarios
6. Unresolved product decisions
7. Verdict: READY or BLOCKED
```

Do not run apply while the verdict is BLOCKED. Correct proposal, specs, design, and tasks first; then repeat the review.

Compliance Matrix inside design.md

Stage	Output	Done when
Project rule	Implementation	Verification
Tenant isolation	Server clinic scope	Cross-clinic API tests
Role permissions	Authorization policy	Forbidden-role tests
Offline idempotency	Stable operation ID	Duplicate retry test
Audit logging	Audit event	Audit integration test

15. Reference - Tests, CI, and Daily Workflow

Minimum executable invariants

- Clinic A cannot read or update Clinic B records.
- Changing clinicId in a request never grants access.
- Unauthorized roles receive a forbidden response.
- Sending the same offline operation twice creates one result.
- Failed operations can be retried safely.
- Financial records cannot be silently deleted.
- Corrections create an audit trail.
- A doctor cannot access another doctor's financial details.

CI merge gate

```
OpenSpec validation
|
Focused unit tests
|
Authorization and tenant tests
|
Integration and offline tests
|
Specification-compliance checklist
|
Merge allowed
```

Daily feature cycle

1. Propose the change.
2. Run the conflict review.
3. If BLOCKED, correct the artifacts and review again.
4. Human approves the specification.
5. Apply the approved tasks.
6. Run executable checks and acceptance scenarios.
7. Review delivered behavior against the spec.
8. Archive only after compliance is proven.

Working principle

Do not try to guarantee agent intelligence. Design the workflow so that a violation becomes visible as a BLOCKED verdict, a missing matrix row, a failing test, or a rejected CI run.